

提示词工程

核心原则：研究表明首尾的提示词对 LLM 的输出结果影响更大

常用提示词框架

1. CRISPE 框架

适用于需要明确角色和任务的场景

- **C (Capacity and Role)** - 角色定位
 - 例: "你是一位资深产品经理"
- **R (Insight)** - 背景洞察
 - 例: "当前市场竞争激烈,用户留存率下降"
- **S (Statement)** - 任务陈述
 - 例: "设计一个用户激励方案"
- **P (Personality)** - 输出风格
 - 例: "用简洁专业的语言呈现"
- **E (Experiment)** - 实验要求
 - 例: "提供 3 个不同方向的方案"

示例:

你是一位资深产品经理(C),当前市场竞争激烈,用户留存率下降(R)。
请设计一个用户激励方案(S),用简洁专业的语言呈现(P),
并提供 3 个不同方向的方案(E)。

2. BROKE 框架

适用于需要深度思考和推理的场景

- **B (Background)** - 背景信息
- **R (Role)** - 角色设定
- **O (Objectives)** - 目标任务
- **K (Key Result)** - 关键结果
- **E (Evolve)** - 迭代优化

示例:

背景: 我正在开发一个微信小程序(B)
角色: 你是前端架构师(R)
目标: 优化首页加载速度(O)
关键结果: 将 FCP 控制在 1.5s 以内(K)
迭代: 如果方案不可行,请提供替代方案(E)

3. COSTAR 框架

适用于复杂任务的系统性处理

- **C (Context)** - 上下文
- **O (Objective)** - 目标
- **S (Style)** - 风格
- **T (Tone)** - 语气
- **A (Audience)** - 受众
- **R (Response)** - 响应格式

示例:

上下文: 公司准备上线新功能(C)
目标: 撰写产品发布文案(O)
风格: 科技感+亲和力(S)
语气: 兴奋且专业(T)
受众: 25-35 岁互联网用户(A)
格式: 标题+3 段正文+CTA(R)

4. RISEN 框架

适用于需要精准输出的场景

- **R (Role)** - 角色
- **I (Input)** - 输入信息
- **S (Steps)** - 执行步骤
- **E (Expectation)** - 期望结果
- **N (Narrowing)** - 约束条件

示例:

角色: 数据分析师(R)
输入: 上季度销售数据(I)
步骤: 1.清洗数据 2.分析趋势 3.提取洞察(S)
期望: 可执行的业务建议(E)
约束: 报告不超过 1 页 A4 纸(N)

5. APE 框架

适用于需要创意和行动方案的场景

- **A (Action)** - 行动指令
- **P (Purpose)** - 目的意图
- **E (Expectation)** - 期望效果

示例:

行动: 撰写一篇技术博客(A)
目的: 帮助初学者理解 React Hooks(P)
期望: 包含代码示例和实践建议,阅读时间 10 分钟(E)

6. RASCEF 框架

适用于结构化输出和专业文档

- **R (Role)** - 角色
- **A (Action)** - 行动
- **S (Steps)** - 步骤
- **C (Context)** - 情境
- **E (Example)** - 示例
- **F (Format)** - 格式

示例:

角色: 技术文档工程师(R)

行动: 编写 API 接口文档(A)
步骤: 按 RESTful 规范组织(S)
情境: 面向第三方开发者(C)
示例: 提供 curl 调用示例(E)
格式: Markdown + OpenAPI 3.0(F)

7. TAG 框架

适用于快速迭代的场景

- **T (Task)** - 任务
- **A (Action)** - 行动
- **G (Goal)** - 目标

示例:

任务: 优化代码性能(T)
行动: 重构核心算法(A)
目标: 响应时间降低 50%(G)

8. RACE 框架

适用于问题解决场景

- **R (Role)** - 角色
- **A (Action)** - 行动
- **C (Context)** - 上下文
- **E (Example)** - 示例

示例:

角色: 系统架构师(R)
行动: 设计分布式缓存方案(A)
上下文: 日活 1000 万用户,高并发场景(C)
示例: 参考 Redis Cluster 的实现(E)

框架选择指南

场景类型	推荐框架	原因
创意生成	CRISPE, APE	灵活性高,适合头脑风暴
技术开发	RISEN, BROKE	结构清晰,适合复杂任务
文案撰写	COSTAR	考虑受众和风格
问题解决	RACE, TAG	目标导向,快速迭代
专业文档	RASCEF	格式规范,结构完整

最佳实践

✓ 提升效果的技巧

- 首尾强化:** 关键信息放在开头和结尾
- 具体化:** 避免模糊表述,提供具体示例
- 分步引导:** 复杂任务拆解为多个步骤
- 约束条件:** 明确字数、格式、风格等限制
- Few-shot 学习:** 提供 2-3 个示例参考

✗ 常见误区

- 过于简短,缺少上下文
- 一次性提出多个不相关任务
- 没有明确输出格式要求
- 忽略角色和受众设定
- 缺少验证和迭代机制

提示词框架 vs 推理技术

◆ 概念区分

类型	定义	代表	作用
提示词框架	组织和构建提示词的结构化模板	CRISPE、BROKE、COSTAR	帮助你写出清晰完整的提示词
推理技术	引导 AI 进行思考和推理的方法	CoT、ToT、GoT	提升 AI 的推理能力和输出质量

主流推理技术

1. CoT - Chain of Thought (链式思考)

核心: 让 AI 逐步展示推理过程

触发方式:

方式 1: 直接指令

请一步步思考这个问题

方式 2: Few-shot 示例

问题: 咖啡店有 23 杯咖啡, 卖出 15 杯, 又做了 18 杯, 现在有多少杯?

思考过程:

1. 初始: 23 杯

2. 卖出后: $23 - 15 = 8$ 杯

3. 新做后: $8 + 18 = 26$ 杯

答案: 26 杯

现在请用同样的方式解决: [你的问题]

适用场景:

- 数学推理
- 逻辑分析
- 复杂决策
- 问题拆解

实际案例:

问题: 如何设计一个高并发秒杀系统?

请一步步分析:

1. 首先识别核心挑战
2. 然后列出技术方案
3. 接着评估每个方案的优缺点
4. 最后给出推荐架构

2. ToT - Tree of Thought (树状思考)

核心: 探索多个可能的推理路径,像下棋一样思考

触发方式:

请探索多个解决方案:

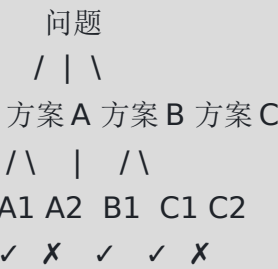
1. 方案 A: [描述] → 评估可行性 → 继续/放弃
2. 方案 B: [描述] → 评估可行性 → 继续/放弃
3. 方案 C: [描述] → 评估可行性 → 继续/放弃

对最可行的方案展开详细设计

适用场景:

- 创意头脑风暴
- 战略规划
- 算法设计
- 多方案对比

图示:



3. GoT - Graph of Thought (图状思考)

核心: 允许思维节点之间相互引用和合并,更灵活的推理网络

触发方式:

请构建一个思维图谱:

1. 列出所有相关因素 (节点)
2. 标注因素之间的关系 (边)
3. 识别关键路径和循环依赖
4. 综合分析给出结论

适用场景:

- 复杂系统分析
- 多因素决策
- 因果关系推理
- 知识图谱构建

示例:

问题: 分析产品增长停滞的原因

请构建因素关系图:

- 用户留存 ← 产品体验, 竞品压力
- 产品体验 ← 功能完整度, 性能问题
- 新增用户 ← 营销投入, 口碑传播
- 口碑传播 ← 用户满意度
- 用户满意度 ← 产品体验, 客服质量

然后分析主要瓶颈在哪里

4. Self-Consistency (自我一致性)

核心: 生成多个推理路径,选择最一致的答案

触发方式:

请用 3 种不同的方法解决这个问题，
然后比较结果,如果一致则输出,如果不一致则分析原因

适用场景:

- 关键决策验证
- 数学题校验
- 逻辑推理

5. ReAct - Reasoning + Acting (推理+行动)

核心: 交替进行推理和行动

触发方式:

请按以下模式解决问题:
思考: [分析当前状态]
行动: [执行一个操作]
观察: [查看结果]
思考: [基于结果继续分析]
行动: [下一步操作]
...

适用场景:

- 工具调用场景
- 多步骤任务
- 交互式问题解决

推理技术对比

技术	复杂度	计算成本	准确性	适用任务
CoT	☆☆	低	高	通用推理
ToT	☆☆☆	中	很高	需要探索多方案
GoT	☆☆☆☆	高	最高	复杂系统分析

Self-Consistency	☆☆☆	高	很高	关键决策
ReAct	☆☆☆	中	高	需要工具调用

实战组合使用

组合 1: CRISPE + CoT

你是一位系统架构师(C),
当前系统面临高并发压力(R),
请设计缓存优化方案(S),
用技术文档格式输出(P),
提供 3 个方案(E)。

请一步步思考:

1. 先分析当前瓶颈
2. 再列出可选技术
3. 然后对比优缺点
4. 最后给出推荐方案

组合 2: RISEN + ToT

角色: 产品经理(R)
输入: 用户调研数据(I)
步骤: 探索 3 种产品方向(S)
期望: 可落地的产品方案(E)
约束: 3 个月内上线(N)

请用树状思考:

- 分支 1: 功能增强路径
 - 分支 2: 用户体验优化路径
 - 分支 3: 新场景拓展路径
- 对每条路径评估可行性

高级技巧

✓ 提升推理质量

1. 显式要求中间步骤

请展示完整的思考过程,不要直接给出答案

1. 使用 **Few-shot** 示例

示例 1: [问题] → [推理过程] → [答案]

示例 2: [问题] → [推理过程] → [答案]

现在请解决: [你的问题]

1. 引入自我验证

得出结论后,请:

1. 检查推理逻辑是否严密
2. 验证计算是否正确
3. 如果发现错误,请重新推理

1. 分解复杂问题

请将问题分解为 3 个子问题,
分别解决后再综合

✗ 常见误区

- 期望一次性解决复杂问题 → 应分步引导
- 不给任何示例 → 应提供 **Few-shot**
- 只要结果不要过程 → 应显式要求推理步骤
- 混淆框架和技术 → 应明确使用场景

角色扮演增强技巧

你是【具体角色】,拥有【专业背景】,
在【具体场景】下,你会如何【具体行动】?
请站在【角色视角】给出建议。

示例:

你是 **Netflix** 的推荐算法工程师,拥有 **10** 年机器学习经验,
在用户兴趣快速变化的场景下,你会如何优化推荐系统?
请站在工程落地的视角给出建议。